

Name of the Student	Guggilla Venkatasai
Reg. No	192425388
GitHub Link	https://github.com/venkatasai-eng/MLA0403-Deep-learning

SIMATS ENGINEERING

CO3 - ASSESSMENT TOOL 1

Case Study

COURSE NAME: DEEP LEARNING COURSE CODE: MLA04 SLOT : A

COURSE OUTCOME COVERED

CO3: Analyze and apply convolutional neural network architectures, including convolutional layers, filters, parameter sharing, regularization, AlexNet and ResNet, to develop solutions for image classification and real-world computer vision applications. (L4)

CO Weightage: 25% **Duration: 90 minutes**

Assessment Tool Weightage: 25% **Marks: 25**

Code of Conduct:

I G.Venkatasai(192425388) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



Name of the student: G.Venkatasai

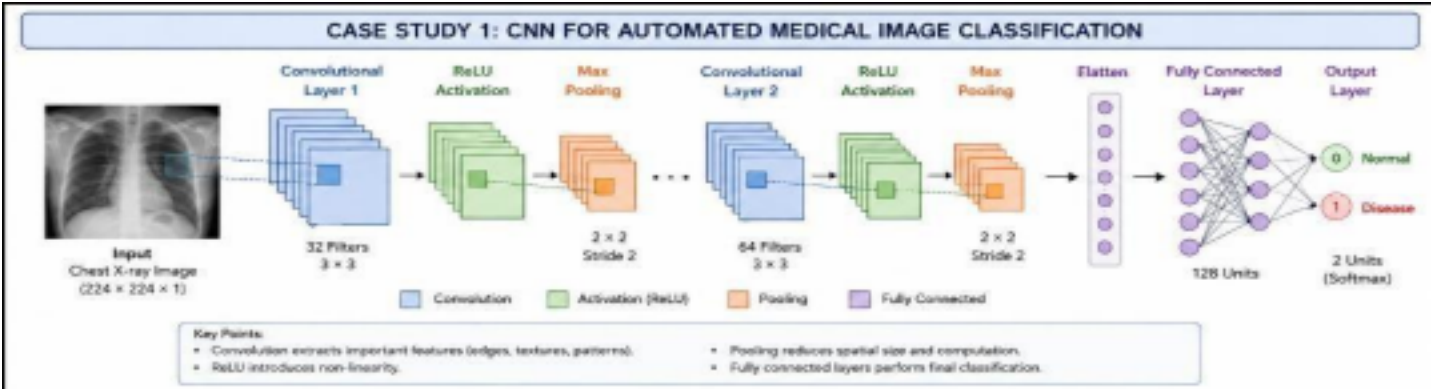
Criteria	Conceptual Understanding (2.5)	Linkages (2.5)	Organization (2)	Integration of Literature (2)	Clarity & Presentation (1)	Total (10)
<i>Weightage</i>	25 %	25%	20%	20 %	10 %	100%
Q1						
Q2						

Total						
-------	--	--	--	--	--	--

Signature of the Faculty: Total Marks :

Case Study Question 1: CNN for Automated Medical Image Classification

A hospital network wants an automated system that sorts chest X-rays into normal and disease-affected categories. Because the images vary in resolution, brightness, and overall quality, a convolutional neural network (CNN) is the natural choice: it learns edges, textures, and abnormal regions directly from pixel data instead of relying on hand crafted rules.



1. Proposed CNN Architecture

Because the diagnostic evidence in an X-ray is spatial — it lives in the arrangement of pixels, not in any single pixel — a CNN is well matched to the task. It builds up understanding in stages: local patterns are detected first, then progressively combined into more abstract, disease-relevant representations. The overall flow is:

Input X-ray → normalization → convolution blocks → pooling → global feature aggregation → dropout → fully connected / output layer.

Figure 1. Proposed CNN pipeline for chest X-ray classification



Figure 1. Proposed CNN pipeline for chest X-ray classification.

2. Why a CNN, and What Convolution and Filters Do

The core assumption behind a CNN is that neighboring pixels are related, and that relationship carries information. A filter slides over the image and produces a response for every local window, generating a feature map. In the first layers, filters typically pick up simple structure — edges and intensity changes. Deeper layers recombine those responses into textures, shapes, and eventually larger anatomical patterns that matter for diagnosis. Learning this hierarchy directly from data is far more flexible than manually coding rules for what a disease 'looks like'.

3. Pooling and the Path to Classification

Pooling shrinks the spatial size of feature maps while keeping the strongest signals. Max pooling in particular cuts computation and gives the network some tolerance to small shifts in the image. That said, pooling too aggressively risks discarding faint but clinically important detail, so the amount of downsampling needs to be deliberate. Global average pooling is a good final step before classification — it condenses each feature map to a single value without the huge parameter cost of flattening into a dense layer.

The output layer then turns those pooled features into a decision. A single sigmoid unit can express the probability of disease, or a two-unit softmax can produce separate normal/disease scores. Either way, the output is a statistical classification that needs clinical review — not an autonomous diagnosis.

4. Parameter Sharing and Why It Matters

Parameter sharing is what makes CNNs practical for image-sized inputs. Consider a 224x224 grayscale image feeding directly into a fully connected layer of 128 neurons — that alone would require $224 \times 224 \times 128 = 6,422,528$ weights. A convolutional layer with 32 filters of size 3x3, by contrast, needs only $3 \times 3 \times 32 = 288$ weights (plus biases), because the same 32 filters are applied at every position in the image.

Figure 2. Parameter sharing drastically cuts trainable weights



Figure 2. Parameter sharing cuts the trainable-weight count by orders of magnitude.

This reuse does two things at once: it keeps memory and training cost low, and it lets a filter that has learned a useful pattern near one part of an X-ray recognize that same pattern elsewhere. The network ends up with reusable visual detectors rather than a separate, position-specific parameter for every pixel.

5. Regularization to Control Overfitting

- Data augmentation: apply small, clinically realistic rotations, translations, scaling, and brightness shifts — transformations that don't distort meaningful anatomy.
- Dropout: randomly deactivate classifier units during training so the network doesn't lean too heavily on a narrow set of features.
- Batch normalization: keep the distribution of intermediate activations stable, which makes optimization more reliable.
- Early stopping: halt training once validation performance plateaus, rather than letting the model memorize the training set.
- Balanced evaluation: track precision, recall, F1-score, and the confusion matrix alongside accuracy, since disease and normal cases are rarely equally represented.

6. AlexNet or ResNet?

AlexNet is historically significant but comparatively shallow, and it leans heavily on large fully connected layers. ResNet is deeper, and its shortcut (skip) connections let gradients and information flow more easily through many layers — which makes deep networks far easier to train and reduces the practical impact of vanishing gradients.

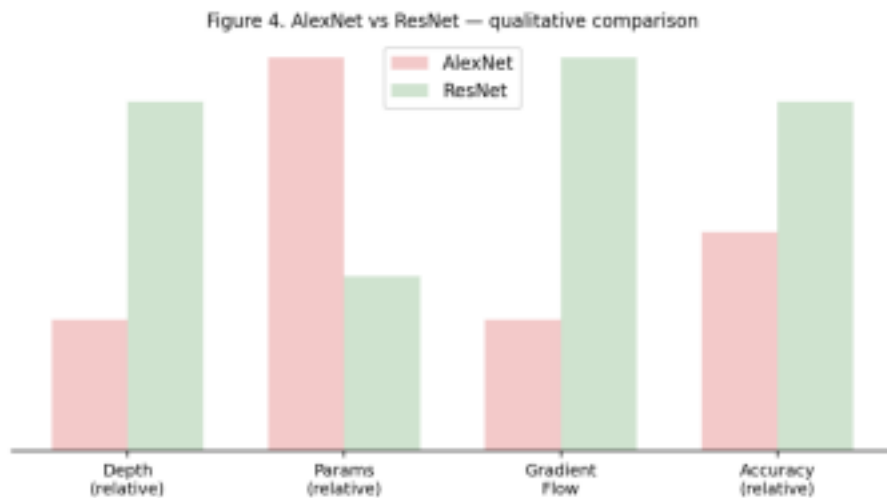


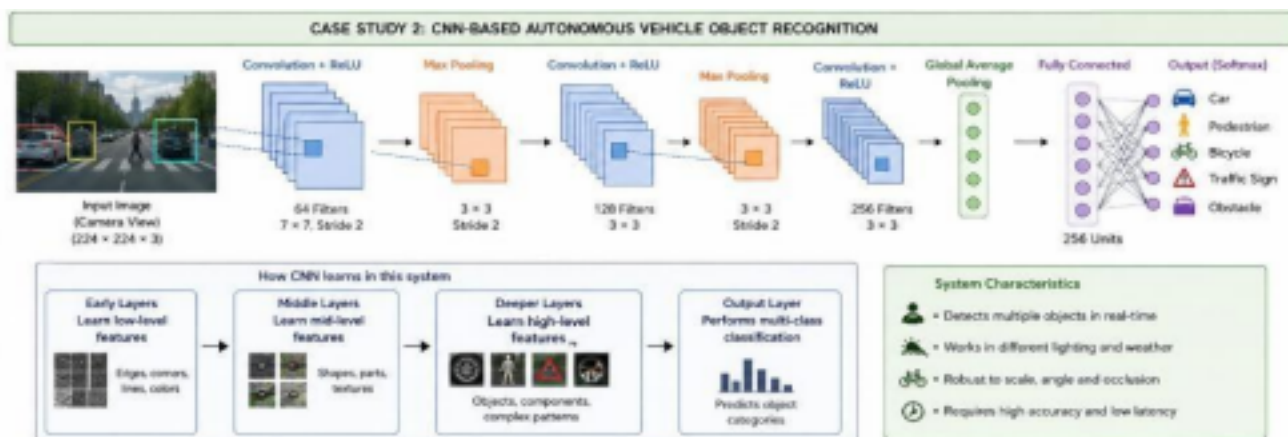
Figure 4. Qualitative comparison of AlexNet and ResNet characteristics.

For this application, ResNet is the stronger choice. Chest X-rays contain subtle, layered patterns that benefit from richer, deeper feature representations, and a compact ResNet variant can deliver that depth without excessive computational overhead. Its residual structure also makes it a strong starting point for transfer learning if a suitable pretrained model is available.

7. Final Design Decision

The recommended pipeline combines standardized X-ray input, residual convolutional feature learning, controlled pooling, global average pooling, dropout, and a binary output layer. Its strengths are automatic feature extraction, efficient parameter use, and better control over overfitting. Because a model can inadvertently learn scanner- or hospital-specific artifacts, independent testing on unseen sources is essential, and the system should be positioned to support — not replace — a qualified clinician's decision.

Case Study Question 2: CNN-Based Autonomous Vehicle Object Recognition An autonomous vehicle company needs a vision system that reliably identifies cars, pedestrians, traffic signs, bicycles, and road obstacles from onboard camera footage — across varying lighting, viewing angles, distances, and road environments. The team is weighing AlexNet against ResNet for the underlying CNN.



1. CNN Architecture for the Vehicle Vision System

Because a vehicle's camera feed constantly changes with lighting, viewpoint, distance, and road conditions, the network needs to recognize the same object regardless of its size or position in the frame. A suitable pipeline

is:

Camera frame → *preprocessing* → *convolution* → *activation / normalization* → *downsampling* → *residual blocks* → *compact feature representation* → *output*.

Figure 3. CNN pipeline for autonomous-vehicle object recognition



Figure 3. CNN pipeline for autonomous-vehicle object recognition.

2. Purpose of the Main Layers

Convolutional filters scan local neighborhoods and output feature maps. Early layers tend to capture low-level structure — edges, corners, and brightness transitions. Middle layers assemble those into parts, such as wheel-like curves, sign borders, or human silhouettes. The deepest layers integrate these parts into full, high-level object representations — vehicles, pedestrians, cyclists, and other obstacles — which the fully connected/output layer then classifies.

3. Parameter Sharing for Real-Time Vision

A road feature — a wheel, a sign edge, a limb — can appear anywhere in the frame from one moment to the next. Parameter sharing means one learned filter is applied across every spatial location instead of training a separate detector per position, which sharply reduces the parameter count compared to a fully connected approach and helps the network keep recognizing an object even as it moves across the frame.

Fewer parameters mean lower memory demands and more predictable computation, which matters for a system that must run in real time. More fundamentally, shared filters learn patterns that generalize across the frame — valuable when cars, pedestrians, and signs are continuously entering and leaving different regions of view.

4. Overfitting and Recommended Regularization

Overfitting is a particularly serious risk in this setting: a model can look accurate on familiar roads while failing under new conditions, for instance by associating a class with a background texture, an exposure setting, or a weather pattern rather than the object itself. The training data should therefore span diverse roads, viewpoints, distances, lighting, and weather.

- Data augmentation: vary brightness, contrast, scale, crop, and translation with realistic transformations; simulate night and weather conditions without destroying object identity.
- Early stopping: monitor performance on a held-out validation environment and stop before the network starts memorizing rather than generalizing.
- Batch normalization and dropout: stabilize training and reduce reliance on any single feature pathway, echoing the same principles used in the medical imaging system above.

5. AlexNet versus ResNet

AlexNet remains historically important but is relatively shallow and depends more on large fully connected layers. ResNet's shortcut connections make much deeper networks trainable, allowing it to capture richer, multi-scale representations of the same object under changing scale and viewpoint — while a compact ResNet variant can still keep the classifier computationally efficient.

6. Final Recommendation

ResNet is the recommended architecture. Its shortcut connections support deep feature learning while convolutional parameter sharing keeps feature extraction efficient enough for real-time use. The complete pipeline is: camera capture → preprocessing → residual CNN → object representation → classification/detection output → confidence-aware vehicle logic. AlexNet remains a useful historical baseline, but ResNet offers a better overall balance of feature learning, gradient flow, and flexible model sizing for this application.